



KnowledgeService.java, 编排知识库构建流程;

```
public class KnowledgeService {  
    public Map<String, Object> buildKnowledge(String repoId, String ownerLogin, String token, boolean ignoredIndexEachCommit,  
                                              int maxCommits, List<String> ignoredCommitShas, String taskId) {  
        String progressKey = "knowledge:" + repoId;  
        progress.start(progressKey, 5, "准备同步仓库");  
        tasks.start(taskId);  
        try {  
            JsonNode repo = authorization.requireAccess(repoId, token);  
            String fullName = required(repo, field: "full_name");  
            String branch = repo.path("default_branch").asText("main");  
            RepoIndex index = store.upsertIndex(repoId, fullName, branch, "indexing", ownerLogin);  
            store.upsertSettings(repoId, false, maxCommits, "");  
  
            tasks.progress(taskId, 1, 5, "克隆或拉取默认分支");  
            progress.step(progressKey, "同步 Git 默认分支");  
            GitRepositoryService.SyncResult sync = git.sync(repoId, fullName, ownerLogin, token, branch);  
            tasks.setCommits(taskId, sync.oldHead(), sync.head());  
  
            String codeWikiId = index.getCodeWikiRepoId() == null ? "" : index.getCodeWikiRepoId();  
            boolean fullBuild = codeWikiId.isBlank();  
            if (fullBuild) {  
                tasks.progress(taskId, 2, 5, "向 CodeWiki 注册本地仓库");  
                CodeWikiClient.RepoResponse registered = codeWiki.register(sync.codeWikiPath(), fullName);  
                codeWikiId = registered == null ? "" : registered.resolvedId();  
                if (codeWikiId.isBlank()) {  
                    codeWikiId = findRegisteredRepo(fullName, sync.codeWikiPath());  
                }  
                if (codeWikiId.isBlank()) throw new IllegalStateException(s: "CodeWiki 注册响应缺少仓库 id");  
                index.setCodeWikiRepoId(codeWikiId);  
                store.saveIndex(index);  
  
                tasks.progress(taskId, 3, 5, "CodeWiki 正在分析源码");  
                CodeWikiClient.RunResponse run = codeWiki.analyze(codeWikiId);  
                if (run == null || run.resolvedId().isBlank()) {  
                    throw new IllegalStateException(s: "CodeWiki analyze 响应缺少 run_id");  
                }  
                codeWiki.awaitRun(codeWikiId, run.resolvedId());  
                tasks.progress(taskId, 4, 5, "构造 GraphRAG");  
                codeWiki.buildGraph(codeWikiId);  
            } else {  
                tasks.progress(taskId, 3, 5, "增量更新 CodeWiki");  
                codeWiki.update(codeWikiId);  
            }  
  
            JsonNode graphStatus = codeWiki.graphStatus(codeWikiId);  
            projectIndex(index, repo, sync.head(), graphStatus);  
            store.upsertSettings(repoId, false, maxCommits, sync.head());  
            tasks.projectCounts(taskId, graphStatus);  
            progress.finish(progressKey, "知识库构建完成");  
            Map<String, Object> quality = tasks.complete(taskId, repoId);  
  
            Map<String, Object> response = new LinkedHashMap<>();  
            response.put(key: "taskId", taskId);  
            response.put(key: "repoId", repoId);  
            response.put(key: "fullName", fullName);  
            response.put(key: "indexedCommits", value: 1);  
            response.put(key: "commits", List.of(Map.of(k1: "commitSha", sync.head(),  
                                                       k2: "shortSha", shortSha(sync.head()), k3: "message", v3: "HEAD")));  
            response.put(key: "activeCommitSha", sync.head());  
            response.put(key: "deduplication", storageStats(repoId));  
            response.put(key: "quality", quality);  
            response.put(key: "status", value: "ready");  
            response.put(key: "mode", fullBuild ? "full" : "incremental");  
            return response;  
        } catch (Exception ex) {  
            store.resetIndexingStatus(repoId);  
            String message = rootMessage(ex);  
            progress.fail(progressKey, message);  
            if (ex instanceof CodeWikiException codeWikiError) {  
                tasks.error(taskId, codeWikiError.operation(), "", "CODEWIKI_ERROR",  
                           message, codeWikiError.retryable());  
            }  
            tasks.fail(taskId, repoId, message);  
            throw ex instanceof IllegalStateException state ? state  
                  : new IllegalStateException("构建知识库失败: " + message, ex);  
        }  
    }  
}
```

KnowledgeQueryService.java，用户提问后 第一个被调用的服务，决定查询方式

```
@Service
public class KnowledgeQueryService {

    private static final Pattern COMMIT_COUNT = Pattern.compile(regex: "(\\d+)\\s*(?:个)?\\s*(?:commit|提交)", Pattern.CASE_INSENSITIVE);
    private static final List<String> API_PATH_HINTS = List.of(
        "openapi", "swagger", "controller", "api/generated");
    private static final List<String> DEPLOY_PATH_HINTS = List.of(
        "readme", "run.ps1", "application.yaml", "application.yml", "package.json",
        "pom.xml", "vite.config", "dockerfile", "docker-compose", ".env");

    private final KnowledgeService knowledgeService;

    public KnowledgeQueryService(KnowledgeService knowledgeService) {
        this.knowledgeService = knowledgeService;
    }

    public QueryResult retrieve(String repoId, String question, String ownerLogin) {
        String lower = question.toLowerCase();
        Set<String> intents = new LinkedHashSet<>();
        if (containsAny(lower, ...terms: "commit", "提交", "版本历史", "提交历史", "弃用", "废弃", "采用", "revert", "回滚")) {
            intents.add(e: "history");
        }
        if (containsAny(lower, ...terms: "api", "接口", "endpoint", "openapi", "swagger")) {
            intents.add(e: "api");
        }
        if (containsAny(lower, ...terms: "部署", "布置", "启动", "安装", "环境变量", "服务器", "docker", "上线",
            "deploy", "deployment", "start", "install", "environment variable")) {
            intents.add(e: "deployment");
        }
        if (containsAny(lower, ...terms: "项目目的", "主要目的", "主要功能", "项目介绍", "做什么", "是什么项目",
            "purpose", "project overview")) {
            intents.add(e: "overview");
        }
        if (intents.isEmpty()) {
            intents.add(e: "code");
        }

        List<Map<String, Object>> contexts = new ArrayList<>();
        try {
            if (intents.contains(o: "history")) {
                contexts.addAll(knowledgeService.commitHistoryContexts(repoId, ownerLogin, requestedCommitCount(question)));
            }
            if (intents.contains(o: "overview")) {
                contexts.add(knowledgeService.repositoryOverviewContext(repoId, ownerLogin));
                contexts.addAll(knowledgeService.retrieveChunksByPathHints(
                    repoId, ownerLogin, question + " 项目说明 README", List.of("readme", "openapi", "package.json", "pom.xml"), 8));
            }
            if (intents.contains(o: "api")) {
                contexts.addAll(knowledgeService.apiSpecificationContexts(repoId, ownerLogin, 100));
                if (contexts.stream().noneMatch(row -> "api_spec".equals(row.get(key: "sourceType")))) {
                    contexts.addAll(knowledgeService.retrieveChunksByPathHints(
                        repoId, ownerLogin, question + " controller endpoint operationId", API_PATH_HINTS, 15));
                }
            }
            if (intents.contains(o: "deployment")) {
                contexts.addAll(knowledgeService.retrieveChunksByPathHints(
                    repoId, ownerLogin, question + " 启动 配置 端口 环境变量", DEPLOY_PATH_HINTS, 15));
            }
            if (intents.contains(o: "code") || contexts.isEmpty()) {
                contexts.addAll(knowledgeService.retrieveChunks(repoId, ownerLogin, question, null, 10));
            }
        } catch (Exception ex) {
            // ...
        }
    }
}
```

CodeWikiClient.java ， 与 CodeWiki 的通信层

```
public RunResponse analyze(String repoId) {
    return post(repo(repoId) + "/analyze", Map.of(k1: "name_communities", v1: false),
        type: RunResponse.class, operation: "analyze");
}

public RunResponse run(String repoId, String runId) {
    return get(repo(repoId) + "/runs/" + runId, type: RunResponse.class, operation: "poll_run");
}

public JsonNode awaitRun(String repoId, String runId) {
    Instant deadline = Instant.now().plusSeconds(Math.max(a: 1, properties.runTimeoutSeconds()));
    while (Instant.now().isBefore(deadline)) {
        RunResponse current = run(repoId, runId);
        String status = current.status() == null ? "" : current.status().toLowerCase();
        if (status.equals(anObject: "completed") || status.equals(anObject: "complete") || status.equals(anObject: "succeeded")
            || status.equals(anObject: "success") || status.equals(anObject: "ready") || status.equals(anObject: "done")) {
            return mapper.valueToTree(current);
        }
        if (status.equals(anObject: "failed") || status.equals(anObject: "error") || status.equals(anObject: "cancelled")) {
            throw new CodeWikiException("poll_run",
                "CodeWiki 分析失败: " + current.errorText(status),
                false, null);
        }
    }
    try {
        Thread.sleep(Math.max(a: 1, properties.runPollSeconds()) * 1000L);
    } catch (InterruptedException ex) {
        Thread.currentThread().interrupt();
        throw new CodeWikiException("poll_run", "等待 CodeWiki 分析时任务被中断", true, ex);
    }
    throw new CodeWikiException("poll_run", "CodeWiki 分析超时", true, null);
}

public JsonNode update(String repoId) {
    return post(repo(repoId) + "/update",
        new UpdateRequest(refresh_chunks: true, name_communities: false, regenerate_... false), JsonNode.class, "update");
}

public JsonNode buildGraph(String repoId) {
    return post(repo(repoId) + "/graphrag/build",
        new GraphBuildRequest(properties.includeEmbeddings()), type: JsonNode.class, operation: "graphrag_build");
}

public JsonNode retrieve(String repoId, String query, int maxHops) {
    return post(repo(repoId) + "/graphrag/retrieve",
        new RetrieveRequest(query, maxHops, properties.includeEmbeddings()),
        type: JsonNode.class, operation: "graphrag_retrieve");
}
```

```
private CodeWikiException mapped(String operation, RestClientException ex) {
    if (ex instanceof RestClientResponseException response) {
        String detail = response.getResponseBodyAsString();
        try {
            JsonNode body = mapper.readTree(detail);
            detail = text(body, field: "detail", text(body, field: "message", detail));
        } catch (Exception ignored) {}
    }
    return new CodeWikiException(operation,
        "CodeWiki " + operation + " 请求失败 (" + response.getStatusCode().value() + "): " + detail,
        response.getStatusCode().is5xxServerError() || response.getStatusCode().value() == 429, ex);
}
return new CodeWikiException(operation, "CodeWiki " + operation + " 不可用: " + ex.getMessage(), true, ex);
}
```

ProgressService.java ， 构建进度追踪以及进度条辅助实现

```

8
9 You, 21小时前 | 1 author (You)
10 @Service
11 public class ProgressService {
12
13     private final ConcurrentHashMap<String, State> states = new ConcurrentHashMap<>();
14
15     public void start(String key, int total, String message) {
16         State state = new State();
17         state.status = "running";
18         state.total = Math.max(total, 1);
19         state.done = 0;
20         state.message = message;
21         states.put(key, state);
22     }
23
24     public boolean isRunning(String key) {
25         State state = states.get(key);
26         return state != null && "running".equals(state.status);
27     }
28
29     public void setTotal(String key, int total) {
30         State state = states.get(key);
31         if (state != null) {
32             state.total = Math.max(total, 1);
33         }
34     }
35
36     public void setDone(String key, int done, String message) {
37         State state = states.get(key);
38         if (state == null) {
39             return;
40         }
41         state.done = Math.min(Math.max(done, 0), state.total);
42         state.message = message;
43     }
44
45     public void step(String key, String message) {
46         State state = states.get(key);
47         if (state == null) {
48             return;
49         }
50         state.done = Math.min(state.done + 1, state.total);
51         state.message = message;
52     }
53
54     public void finish(String key, String message) {
55         State state = states.get(key);
56         if (state == null) {
57             return;
58         }
59         state.status = "done";
60         state.done = state.total;
61         state.message = message;
62     }
63
64     public void fail(String key, String message) {
65         State state = states.computeIfAbsent(key, k -> new State());
66         state.status = "error";
67         state.message = message;
68     }
69
70     public void idle(String key) {
71         states.remove(key);
72     }
73
74     public Map<String, Object> snapshot(String key) {
75         State state = states.get(key);
76         if (state == null) {
77             return Map.of(
78                 k1: "status", v1: "idle",
79                 k2: "progress", v2: 0,
80                 k3: "message", v3: "",
81                 k4: "total", v4: 0,

```

sdk.gen.ts ， 自动生成的前端 API 契约

```
export type Options<TData extends TDataShape, ThrowOnError extends boolean = boolean, TResponse = unknown> =  
  /**  
   * You can provide a client instance returned by `createClient()` instead of  
   * individual options. This might be also useful if you want to implement a  
   * custom client.  
   */  
  client?: Client;  
  /**  
   * You can pass arbitrary values through the `meta` object. This can be  
   * used to access values that aren't defined as part of the SDK function.  
   */  
  meta?: keyof ClientMeta extends never ? Record<string, unknown> : ClientMeta;  
};  
  
/**  
 * Start GitHub OAuth login  
 *  
 * Browser navigation (not apiFetch). Redirects to GitHub authorize, then backend completes via `/auth/callback` and returns  
 */  
export const startGithubLogin = <ThrowOnError extends boolean = true>(options?: Options<StartGithubLoginData, ThrowOnError>): Promise<TResponse> {  
  /**  
   * Complete GitHub OAuth callback  
   *  
   * GitHub redirects the browser here. The backend validates state, exchanges the code, then redirects to the frontend success page  
   */  
  export const completeGithubLogin = <ThrowOnError extends boolean = true>(options?: Options<CompleteGithubLoginData, ThrowOnError>): Promise<TResponse> {  
    /**  
     * Get current GitHub user profile  
     */  
    export const fetchUserProfile = <ThrowOnError extends boolean = true>(options?: Options<FetchUserProfileData, ThrowOnError>): Promise<TResponse> {  
      responseType: 'json',  
      security: [{ scheme: 'bearer', type: 'http' }],  
      url: '/api/user/profile',  
      ...options  
    });  
  });  
  
  /**  
   * Get LLM configuration  
   */  
  export const fetchLlmConfig = <ThrowOnError extends boolean = true>(options?: Options<FetchLlmConfigData, ThrowOnError>): Promise<TResponse> {  
    responseType: 'json',  
    security: [{ scheme: 'bearer', type: 'http' }],  
    url: '/api/user/setting/llmconfig',  
    ...options  
  });  
  
  /**  
   * Save LLM configuration  
   */  
  export const setLlmConfig = <ThrowOnError extends boolean = true>(options: Options<SetLlmConfigData, ThrowOnError>): Promise<TResponse> {  
    responseType: 'json',  
    security: [{ scheme: 'bearer', type: 'http' }],  
    url: '/api/user/setting/llmconfig/set',  
    ...options,  
    headers: {  
      'Content-Type': 'application/json',  
      ...options.headers  
    }  
  });  
  
  /**  
   * List repositories for the authenticated user  
   */  
  export const fetchRepoList = <ThrowOnError extends boolean = true>(options?: Options<FetchRepoListData, ThrowOnError>): Promise<TResponse> {  
    responseType: 'json',  
  });  
}
```

目录结构

feature/byLiang

14 Branches

0 Tags

Go to file

Add file

Code

This branch is 10 commits ahead of main.

#16

Yu-Liang-Yan

fix: ensureDefaultAdmin() 添加 @eventlistener 确保启动时自动创建管理员账号

7916fe1 · 20 hours ago

30 Commits

UIPrototype	feat(fix): Adjust refs in UIPrototype/README.md	2 weeks ago
backend	fix: ensureDefaultAdmin() 添加 @eventlistener 确保启动时自...	20 hours ago
contract	feat:增加和完善codewiki、graphRAG、AST树和相关接口实现...	yesterday
frontend	feat: 统一用户体系 — app_users成为唯一用户中心	20 hours ago
scripts	feat:增加和完善codewiki、graphRAG、AST树和相关接口实现...	yesterday
services/codewiki	feat:增加和完善codewiki、graphRAG、AST树和相关接口实现...	yesterday
.gitignore	feat:增加和完善codewiki、graphRAG、AST树和相关接口实现...	yesterday
README.md	feat:增加和完善codewiki、graphRAG、AST树和相关接口实现...	yesterday
docker-compose.yml	feat:增加和完善codewiki、graphRAG、AST树和相关接口实现...	yesterday
start-dev.ps1	fix: start-dev.ps1 自动加载 backend\env 环境变量	yesterday

也有 ASCII 树版本

```
SEProject26-7-fresh/
├── frontend/                                     # Vue/React前端
│   └── src/
│       ├── api/generated/                       # 自动生成的SDK（前后端契约）
│       │   ├── sdk.gen.ts                       # API调用函数
│       │   └── types.gen.ts                     # TypeScript类型定义
│       ├── components/                         # 可复用UI组件
│       ├── pages/                             # 页面
│       │   ├── Knowledge.tsx                   # 知识库主页面（构建/进度/问答）
│       │   ├── RepoDetail.tsx                 # 仓库详情页
│       │   └── admin/                         # 管理员页面
│       └── styles/                             # 样式文件
├── backend/                                     # Spring Boot后端
│   └── src/main/java/com/repopilot/
│       ├── client/
│       │   └── CodeWikiClient.java             # CodeWiki HTTP客户端
│       ├── config/
│       │   └── AppProperties.java              # 配置绑定（LLM/CodeWiki/GitHub）
│       ├── controller/
│       │   ├── KnowledgeController.java        # 知识库API入口
│       │   └── ChatController.java            # 问答API入口
│       ├── entity/
│       │   └── RepoIndex.java                  # 仓库索引实体（H2映射）
│       ├── repository/
│       │   └── support/
│       │       └── KnowledgeStore.java          # 数据访问层
│       ├── security/                          # 认证与授权（JWT + GitHub OAuth）
│       └── service/
│           ├── KnowledgeService.java           # ★ 构建编排引擎
│           ├── KnowledgeQueryService.java      # ★ 意图识别路由
│           ├── KnowledgeBuildTaskService.java  # 任务生命周期管理
│           ├── ProgressService.java            # ★ 进度追踪
│           ├── GitRepositoryService.java       # JGit仓库管理
│           └── RepoAuthorizationService.java   # 权限验证
├── services/
│   └── codewiki/
│       └── Dockerfile                          # CodeWiki Docker配置
├── docker-compose.yml                          # 容器编排
└── start-dev.ps1                              # 一键启动脚本
```